Phase 1 to Phase 7 Report
Environment Setup, Firmware Skeleton, Virtual Sensor, Sampling Benchmark, Frequency Analysis, Adaptive Sampling, and Aggregate Computation

IoT Adaptive Sampling Project

April 2026

1 Introduction

This document reports the work completed in **Phase 1**, **Phase 2**, **Phase 3**, **Phase 4**, **Phase 5**, **Phase 6**, and **Phase 7** of the project.

The purpose of Phase 1 was not to implement the full IoT application yet, but to verify that the development environment, the ESP32 board, and the MQTT communication tools were working correctly.

The purpose of Phase 2 was to convert the initial project structure into a real firmware skeleton that boots on the target board and clearly separates the main modules of the final system.

The purpose of Phase 3 was to replace the placeholder input with a real virtual signal generator and verify that complete sample windows could be collected and validated on the board.

The purpose of Phase 4 was to measure the highest stable sampling rate that the current firmware and board could sustain under a strict timing criterion.

The purpose of Phase 5 was to replace the frequency-analysis placeholder with a real spectral computation so that the dominant component of the generated signal could be identified from each completed window.

The purpose of Phase 6 was to use the detected dominant frequency to change the sampling rate at runtime while keeping later windows clean and internally consistent.

The purpose of Phase 7 was to convert each completed analysis window into a real aggregate result so that the average value, current sampling rate, and dominant frequency are available for later communication stages.

The main goal of these early phases was to answer the following practical questions:

> Can the hardware and software environment already support firmware upload, serial monitoring, and MQTT communication before the real project logic is implemented, can the project boot as a structured modular firmware before the real sensing and FFT logic are added, can it generate the required virtual signal, and what is the highest stable sampling rate observed in the current implementation, and can the dominant spectral component be detected correctly from the measured signal windows, and can the system safely adapt

its sampling rate based on that detected frequency, and can each completed window now produce a clean aggregate summary for later transmission?

These phases are important because they reduce technical risk. If the environment and firmware structure are not validated first, later problems with FFT, adaptive sampling, or network communication may be difficult to diagnose.

2 Goal of Phase 1

The objective of Phase 1 was to validate five basic requirements:

1. the ESP32 board is detected by the computer,
2. a simple firmware example can be built and uploaded,
3. the serial monitor shows correct runtime logs,
4. the MQTT broker is running correctly,
5. MQTT communication works both locally and over the local network.

If all five checks pass, the environment is considered ready for the next development phase.

3 Hardware and Software Used

3.1 Hardware

The hardware used in this phase is:

- **RUIZHI ESP32 V3 LoRa Board**
- USB connection to the development computer
- integrated WiFi interface
- integrated LoRa radio, not used yet in Phase 1

3.2 Software

The software and tools used in this phase are:

- **PlatformIO** with the `ESP-IDF` framework
- **Mosquitto** as MQTT broker
- **PowerShell** on Windows
- serial monitor through PlatformIO

3.3 Network Information

The local network address used during testing was:

- **Broker IP:** `192.168.1.22`
- **MQTT Port:** `1883`

4 Smoke Test Project Created for Phase 1

To keep the first test simple and reliable, a small smoke test project was created. Its only purpose was to prove that firmware could be built, uploaded, and executed successfully.

The test project uses:

- PlatformIO project configuration for the board
- `ESP-IDF` as the framework
- a minimal `app_main()` function
- repeated serial output and automatic restart

The board target used in the test project is:

```
[env:heltec_wifi_lora_32_V3]
platform = espressif32
framework = espidf
board = heltec_wifi_lora_32_V3
monitor_speed = 115200
```

This configuration was chosen because it matches the ESP32-S3 based Heltec compatible board used in the project.

5 Detailed Step-by-Step Procedure

5.1 Step 1: Detect the ESP32 Board

The first step was to verify whether Windows could correctly detect the board when it was connected through USB.

Why this step matters

If the operating system does not detect the board correctly, then firmware upload and serial monitoring cannot work.

What was checked

The connected serial devices were inspected in Windows. The relevant result was:

- `COM3` was detected as `Silicon Labs CP210x USB to UART Bridge`

Interpretation

This result confirms that the ESP32 development board is visible to the computer and that the USB-to-serial interface driver is working correctly.

Result

Status: Passed

5.2 Step 2: Build a Minimal Firmware Example

After confirming the board was detected, the next step was to build a very small firmware application.

Why this step matters

This step validates:

- the compiler toolchain,
- the board definition,
- the ESP-IDF integration,
- the project configuration.

What was implemented

A simple firmware program was created with the following behavior:

- print a greeting message,
- print chip information,
- print free heap memory,
- count down from 5 seconds,
- restart automatically.

Key command used

```
& "$HOME\.platformio\penv\Scripts\pio.exe" run
```

Interpretation

The successful build showed that the local PlatformIO and ESP-IDF environment were correctly set up for this board.

Result

Status: Passed

5.3 Step 3: Upload the Firmware to the Board

Once the project compiled successfully, the firmware was uploaded to the board.

Why this step matters

Successful upload proves that:

- the correct serial port is selected,
- the bootloader flashing process works,
- the board can accept new firmware.

Key command used

```
& "$HOME\.platformio\penv\Scripts\pio.exe" run -t upload --upload-port COM3
```

Interpretation

Since the test firmware ran correctly after flashing, the upload process was successful.

Result

Status: Passed

5.4 Step 4: Verify Runtime Through the Serial Monitor

After uploading, the serial monitor was opened to confirm that the board was actually running the expected program.

Why this step matters

This step confirms:

- the board boots correctly,
- the uploaded application starts correctly,
- runtime logs can be observed,
- future debugging will be possible.

Observed output

The following output was observed:

```
Hello world from Stage 1!
This is esp32s3 with 2 CPU core(s), WiFi/BLE, silicon revision v0.2,
8MB external flash
Minimum free heap size: 388752 bytes
Stage 1 smoke test restart in 5 seconds...
Stage 1 smoke test restart in 4 seconds...
Stage 1 smoke test restart in 3 seconds...
Stage 1 smoke test restart in 2 seconds...
Stage 1 smoke test restart in 1 seconds...
Stage 1 smoke test restart in 0 seconds...
Restarting now.
```

Additional boot logs also showed:

- target chip: `esp32s3`
- flash size: `8MB`
- ESP-IDF version: `5.5.3`

Interpretation

This output confirms that:

- the firmware was uploaded correctly,
- the application executed correctly,
- the serial monitor configuration was correct,
- the board information was consistent with the expected hardware.

Result

Status: Passed

5.5 Step 5: Start the MQTT Broker

The next objective was to verify the MQTT side of the environment.

Why this step matters

The project requires sending aggregate values from the ESP32 to a nearby edge server using MQTT over WiFi. Before implementing this in firmware, the broker must be available and working.

Initial command used

```
& "C:\Program Files\mosquitto\mosquitto.exe" -v
```

Initial result

The broker did not start in that terminal because port `1883` was already in use.

Interpretation

This was not a real failure. It meant that a Mosquitto instance was already running on the machine. Further inspection showed that Mosquitto was already installed and active as a Windows service.

Result

Status: Passed

5.6 Step 6: Verify Local MQTT Publish/Subscribe

After confirming that Mosquitto was already running, the broker was tested locally on the same machine.

Why this step matters

This verifies that the broker is operational and can route MQTT messages correctly between a publisher and a subscriber.

Commands used

Subscriber:

```
& "C:\Program Files\mosquitto\mosquitto_sub.exe" -h localhost -t stage1/test -v
```

Publisher:

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -h localhost -t stage1/test -m
    "hello from stage1"
```

Observed result

```
stage1/test hello from stage1
```

Interpretation

This result proves that:

- the broker is alive,
- a client can subscribe to a topic,
- another client can publish to the same topic,
- the broker routes the message correctly.

Result

Status: Passed

5.7 Step 7: Make MQTT Reachable Over the Local Network

At this point, MQTT worked only on `localhost`. However, the real project requires the ESP32 board to connect over WiFi using the local network.

Why this step matters

If Mosquitto only listens on `localhost`, then the development computer can use it, but the ESP32 cannot reach it over WiFi.

Problem observed

Mosquitto was running in *local only mode*. This means it accepted connections only from the local machine.

Configuration change applied

The broker configuration file was updated so that Mosquitto listens on all IPv4 interfaces:

```
listener 1883 0.0.0.0
protocol mqtt
socket_domain ipv4
listener_allow_anonymous true
```

Additional note

The service restart required administrative privileges, and it was also necessary to ensure the configuration lines were not commented out.

Interpretation

This change made the broker visible not only to the local machine, but also to devices on the same local network, including the ESP32 board.

5.8 Step 8: Verify MQTT Over LAN

The final Phase 1 check was to verify MQTT communication using the LAN address of the development computer instead of `localhost`.

Why this step matters

This test is the closest software-side approximation to the future ESP32 communication scenario.

Commands used

Subscriber:

```
& "C:\Program Files\mosquitto\mosquitto_sub.exe" -h 192.168.1.22 -t stage1/test
    -v
```

Publisher:

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -h 192.168.1.22 -t stage1/test
    -m "hello over lan"
```

Observed result

```
stage1/test hello over lan
```

Interpretation

This confirms that:

- the broker is listening on the LAN,
- the chosen port is correct,
- the network path is working,
- the ESP32 will later be able to use the same broker address from firmware.

Result

Status: Passed

6 Problems Encountered and How They Were Solved

During Phase 1, a few small setup issues appeared. They were useful because they helped confirm the behavior of the system.

6.1 Problem 1: Port 1883 Already in Use

Observation: when Mosquitto was started manually, it reported that port `1883` was already in use.

Cause: Mosquitto was already running as a Windows service.

Solution: instead of starting a second broker, the existing service was used.

6.2 Problem 2: Service Restart Failed

Observation: restarting the service from PowerShell failed.

Cause: administrative privileges were required.

Solution: the restart had to be performed from an Administrator PowerShell or from the Windows services panel.

6.3 Problem 3: Listener Configuration Did Not Take Effect

Observation: the broker remained local-only after editing the config.

Cause: the new configuration lines were still commented out with the # character.

Solution: the comment characters were removed and the service was restarted.

7 Summary of Phase 1 Results

Check	Status
ESP32 board detected by Windows on COM3	Passed
Minimal firmware built successfully	Passed
Firmware uploaded successfully	Passed
Serial monitor showed correct runtime logs	Passed
Mosquitto broker available on the computer	Passed
MQTT publish/subscribe worked on localhost	Passed
MQTT publish/subscribe worked over LAN IP `192.168.1.22`	Passed

8 Phase 2: Firmware Skeleton

8.1 Goal of Phase 2

The goal of Phase 2 was to build the first real version of the firmware architecture for the project.

At this stage, the objective was still not to implement the final sensing logic. Instead, the purpose was to create a clean and testable application structure with separate modules and FreeRTOS tasks.

In practical terms, Phase 2 had to answer this question:

> Can the firmware boot as a modular application on the ESP32 board, with all planned subsystems initialized and running as placeholder tasks?

8.2 Expected Result of Phase 2

Phase 2 was considered complete if the following conditions were satisfied:

1. the firmware project compiles successfully,
2. the application boots correctly on the ESP32 board,
3. the modules are separated into their own source files,
4. the FreeRTOS tasks are created correctly,
5. startup logs confirm that all modules are initialized,
6. a supervisor log confirms that the complete structure is active.

8.3 Project Structure Implemented

The firmware project was created under the directory:

- `firmware/esp32_node`

The Phase 2 implementation included:

- a PlatformIO configuration file,
- top-level ESP-IDF CMake files,
- shared project configuration,
- shared project data types,
- one source module for each planned subsystem,
- a central `app_main()` entry point.

The firmware modules created in this phase are:

- signal input
- signal processing
- sampling control
- MQTT communication
- LoRaWAN communication
- metrics
- display

8.4 Why This Structure Was Chosen

The final project will contain multiple responsibilities:

- sample acquisition
- FFT-based analysis
- adaptive sampling control
- MQTT transmission
- LoRaWAN transmission
- measurement collection
- optional display output

Putting all of these responsibilities into a single file would make the code harder to maintain and harder to debug.

For this reason, the firmware was divided into modules from the beginning. This allows future phases to add real logic without changing the overall architecture.

8.5 Shared Data Structures Created

To allow the modules to communicate with each other, shared structures were defined in the common project headers.

The most important data structures introduced in Phase 2 are:

- `raw_sample_t`
- `fft_result_t`
- `aggregate_result_t`
- `transmission_payload_t`
- `timing_metrics_t`
- `project_context_t`

Purpose of each structure

- `raw_sample_t`: stores one generated or measured sample together with timestamp and source information.
- `fft_result_t`: stores the dominant frequency result for one signal window.
- `aggregate_result_t`: stores the average and metadata for a completed window.
- `transmission_payload_t`: stores prepared payloads for MQTT or LoRaWAN communication.
- `timing_metrics_t`: stores counters and timestamps useful for system measurement.
- `project_context_t`: contains the queues, event group, mode, and shared metrics of the whole application.

8.6 FreeRTOS Communication Model

Phase 2 also introduced the basic communication model between tasks.

The following FreeRTOS objects were created:

- sample queue
- FFT queue
- aggregate queue
- MQTT queue
- LoRaWAN queue
- one event group for module readiness flags

Why queues were chosen

Queues were selected because they provide a clean and safe way to pass data between tasks. They are especially suitable for this project because the final system naturally has a pipeline:

1. generate or read samples,
2. process a signal window,
3. adapt the sampling logic,
4. prepare communication payloads,
5. transmit results.

This makes queues a natural fit for the project design.

Why event groups were chosen

The event group was introduced to track whether each subsystem had initialized correctly. This makes the startup process easier to observe and later makes it possible to add stronger supervision logic if needed.

8.7 Main Firmware Entry Point

The central application entry point was implemented in `app_main()`.

The responsibilities of `app_main()` in Phase 2 are:

- print startup information,
- create the shared project context,
- create the FreeRTOS queues,
- create the event group,
- initialize every module,
- create one task per module,
- print a periodic supervisor heartbeat.

8.8 Placeholder Behavior Implemented in Each Module

At this phase, each module contains only placeholder logic. This is intentional. The goal is to prove that the architecture works before adding real sensing and processing.

8.8.1 Signal Input Module

The signal input module currently:

- initializes correctly,
- sets its ready flag,
- periodically creates a placeholder sample,
- pushes the sample into the sample queue.

The placeholder sample currently has:

- source = virtual sensor
- value = 0.0
- a valid sample id
- a real timestamp

8.8.2 Signal Processing Module

The signal processing module currently:

- receives placeholder samples from the sample queue,
- logs that it received them,
- creates a placeholder FFT result every four samples,
- sends that result to the FFT queue.

No real FFT is performed yet. That work belongs to a later phase.

8.8.3 Sampling Control Module

The sampling control module currently:

- waits for FFT results,
- logs a placeholder adaptive decision,
- confirms that the control stage is wired correctly.

No real adaptive policy is applied yet.

8.8.4 MQTT Module

The MQTT module currently:

- initializes with the configured broker address,
- prepares placeholder JSON messages,
- pushes them into the MQTT queue,
- logs the prepared payload.

At this phase it does not yet open a WiFi or broker connection.

8.8.5 LoRaWAN Module

The LoRaWAN module currently:

- initializes correctly,
- prepares a placeholder uplink payload,
- pushes it into the LoRaWAN queue,
- logs the operation.

No real TTN communication is implemented yet.

8.8.6 Metrics Module

The metrics module currently:

- initializes the shared timing structure,
- updates counters and timestamps,
- prints periodic metrics heartbeats.

8.8.7 Display Module

The display module currently:

- initializes correctly,
- logs a periodic heartbeat,
- confirms that the display subsystem is included in the architecture.

In Phase 2 it is only a serial placeholder and does not yet use the OLED.

8.9 Build Validation

After implementing the firmware skeleton, the project was compiled locally.

Why this step matters

This build validation confirms:

- the project structure is correct,
- the headers and components are connected correctly,
- the application is syntactically valid,
- the firmware can be built for the target board.

Result

The firmware built successfully using PlatformIO and the ESP-IDF framework.

An additional configuration improvement was made during this phase:

- the flash size configuration was updated from `2MB` to `8MB` so that it matches the actual board hardware.

8.10 Runtime Validation on the Board

After compiling the firmware, it was uploaded to the ESP32 and observed through the serial monitor.

Why this step matters

This is the most important validation of Phase 2 because it proves that the modular application does not only compile, but also runs correctly on the actual board.

Observed startup log

The runtime log showed the following important lines:

```
I (...) app_main: Phase 2 firmware skeleton booting for adaptive-sampling-node
I (...) app_main: Planned MQTT endpoint: 192.168.1.22:1883
I (...) metrics: Metrics module initialized
I (...) signal_input: Signal input module ready in placeholder mode
I (...) signal_processing: Signal processing module ready with FFT placeholder
    hooks
I (...) sampling_control: Sampling control module ready with adaptive-policy
    placeholder
I (...) comm_mqtt: MQTT module ready for broker 192.168.1.22:1883
I (...) comm_lorawan: LoRaWAN module ready with TTN uplink placeholder
I (...) app_display: Display module ready with serial-only placeholder
I (...) app_main: Phase 2 tasks created successfully
```

The periodic runtime also showed:

```
I (...) app_main: Supervisor heartbeat | events=0xFF samples=1 windows=0 mqtt=1
    lora=1
I (...) signal_input: Queued placeholder sample id=0 source=0 value=0.00
I (...) signal_processing: Received sample id=0 for future FFT pipeline
```

```
I (...) comm_mqtt: Prepared placeholder MQTT payload: {"phase":2,"state":"
    skeleton","broker":"192.168.1.22","port":1883}
I (...) comm_lorawan: Prepared placeholder LoRaWAN payload
I (...) metrics: Metrics heartbeat | ...
I (...) app_display: Display heartbeat | mode=0 events=0xFF
```

Interpretation

These logs confirm that:

- the firmware booted correctly,
- the project configuration was correct,
- all modules initialized successfully,
- all readiness bits were set,
- all FreeRTOS tasks were created,
- placeholder data moved through the intended pipeline.

The value `events=0xFF` is particularly important because it means all module-ready flags were set, which is exactly what was expected from the Phase 2 skeleton.

8.11 What Was Tested in Phase 2

The Phase 2 test was intentionally light and structural. It did not attempt to validate the final algorithm yet.

The following aspects were tested:

- successful build,
- successful upload,
- board boot without crashes,
- module initialization logs,
- recurring supervisor heartbeat,
- recurring placeholder task activity.

The following aspects were **not** tested yet:

- real sinusoidal sample generation,
- FFT correctness,
- real adaptive sampling,
- real WiFi connection,
- real MQTT publishing,
- real LoRaWAN transmission.

8.12 Issue Observed During Phase 2

The runtime logs showed that the MQTT and LoRaWAN queue counts grow slowly over time.

Cause

This happens because, in Phase 2, the communication tasks only prepare placeholder payloads and push them into their queues. They do not yet consume or transmit those payloads.

Interpretation

This is acceptable in Phase 2 because the goal of the phase is architectural validation, not complete communication behavior.

Future fix

This will be resolved in later phases when the real MQTT and LoRaWAN logic is implemented.

8.13 Summary of Phase 2 Results

Check	**Status**
PlatformIO and ESP-IDF project structure created	Passed
Shared queues and event group created successfully	Passed
All modules compiled successfully	Passed
Firmware uploaded successfully to the board	Passed
All module startup logs observed on the serial monitor	Passed
Supervisor heartbeat observed	Passed
Placeholder data moved through the initial pipeline	Passed
No crash, panic, or watchdog reset observed	Passed

9 Phase 3: Virtual Sensor Implementation

9.1 Goal of Phase 3

The goal of Phase 3 was to replace the placeholder sample generation logic with the first real signal used by the project.

At this stage, the firmware had to start producing a deterministic periodic signal directly on the ESP32 board. This signal would then be used in later phases for FFT analysis, adaptive sampling, and aggregate computation.

The practical question of Phase 3 was:

> Can the ESP32 generate the required synthetic sensor signal at a stable sampling rate and store it in a real processing window?

9.2 Required Signal

According to the project requirements, the input signal is of the form:

$$\sum a_k \sin(2\pi f_k t)$$

For the first implementation, the chosen signal was:

$$2\sin(2\pi \cdot 3t) + 4\sin(2\pi \cdot 5t)$$

This signal was selected because:

- it is simple and deterministic,
- it contains more than one frequency component,
- it is easy to validate visually and numerically,
- it is appropriate for later FFT-based frequency analysis.

9.3 What Was Implemented in Phase 3

The Phase 3 implementation added three main capabilities:

1. real virtual signal generation,
2. stable periodic sampling,
3. window-based validation of generated samples.

9.4 Configuration Added

The firmware configuration was extended with signal-specific parameters.

The main configuration values introduced in this phase were:

- initial sampling frequency: `50 Hz`
- signal term 1 amplitude: `2.0`
- signal term 1 frequency: `3 Hz`
- signal term 2 amplitude: `4.0`
- signal term 2 frequency: `5 Hz`
- window duration: `5 seconds`
- expected window size: `250 samples`

Why 50 Hz was used

The initial sampling rate of `50 Hz` was chosen because:

- it is comfortably above the Nyquist limit for the highest component of `5 Hz`,
- it gives a stable first implementation,
- it makes the signal easy to inspect before testing aggressive sampling rates in Phase 4.

9.5 Signal Input Module Changes

The signal input module was updated so that it no longer produces a constant placeholder value.

Instead, it now:

- computes the elapsed time from system start,
- evaluates the sinusoidal signal formula,
- creates timestamped samples,
- pushes those samples to the sample queue,
- runs periodically using `vTaskDelayUntil()` for more stable timing.

Why `vTaskDelayUntil()` was used

This function is preferable to a simple delay because it keeps the task closer to a fixed periodic schedule. This is important for signal processing because sampling should be as regular as possible.

9.6 Signal Processing Module Changes

The signal processing module was extended from a simple placeholder receiver into a basic validation stage.

Its responsibilities in Phase 3 are:

- receive generated samples,
- collect them into a fixed-size window,
- print the first few sample values of each window,
- compute simple statistics for the completed window,
- prepare a placeholder FFT result for the next stage.

Why this validation stage matters

Before implementing FFT, it is important to verify that:

- the samples are not constant,
- the values evolve over time as expected,
- the window fills correctly,
- the per-window statistics are reasonable.

9.7 Additional Cleanup During Phase 3

A small structural improvement was also introduced in the communication placeholder modules.

The MQTT and LoRaWAN placeholder tasks were updated so that their queues keep only the most recent placeholder messages instead of growing without bound during long runs.

Why this change was useful

This change was not the main objective of Phase 3, but it helps keep long test runs cleaner while the communication modules are still placeholders.

9.8 Build Validation

After the Phase 3 changes were implemented, the firmware was compiled again.

Why this matters

This confirms that:

- the new signal generation code is syntactically correct,
- the mathematical functions are linked correctly,
- the updated sampling and windowing logic integrates with the existing firmware architecture.

Result

The firmware compiled successfully.

9.9 Runtime Validation on the Board

After the build, the updated firmware was uploaded and monitored on the ESP32 board.

Observed startup log

The following key startup lines were observed:

```
I (...) app_main: Phase 3 virtual sensor firmware booting for adaptive-sampling-
    node
I (...) signal_input: Signal input module ready in virtual-sensor mode at 50.0
    Hz
I (...) signal_processing: Signal processing module ready with 256-sample
    validation window
```

Interpretation

These startup lines confirm that the firmware entered the correct execution mode and that the virtual signal path was active.

9.10 Observed Sample Values

The first samples of the first processing window were logged as:

```
Window 0 sample[0] = 0.0008
Window 0 sample[1] = 2.2757
Window 0 sample[2] = 4.7317
Window 0 sample[3] = 5.6716
Window 0 sample[4] = 4.8578
```

Interpretation

These values are very important because they prove that:

- the signal is no longer a placeholder constant,
- the values change continuously over time,
- the waveform is behaving like a real sum of sinusoids.

9.11 Observed Sampling Behavior

During execution, sample-generation logs also appeared periodically:

```
Generated sample id=25  t=0.494s  value=0.9205
Generated sample id=50  t=0.994s  value=-0.9205
Generated sample id=75  t=1.494s  value=0.9205
Generated sample id=100 t=1.994s  value=-0.9205
```

Interpretation

This confirms that:

- the task is sampling periodically,
- the timestamps progress correctly,

- the signal alternates as expected over time,
- the generator remains stable across repeated intervals.

9.12 Observed Window Completion

The end of the first completed window produced the following summary:

```
Window 0 ready | samples=250 avg=0.0037 min=-5.6716 max=5.6716 duration=4.97s
```

Subsequent windows showed similar results, for example:

```
Window 1 ready | samples=250 avg=-0.0000 min=-5.6716 max=5.6716 duration=4.98s
Window 2 ready | samples=250 avg=0.0000 min=-5.6716 max=5.6716 duration=4.98s
Window 3 ready | samples=250 avg=0.0000 min=-5.6716 max=5.6716 duration=4.98s
```

Interpretation

These results confirm several important points:

- each window contains the expected number of samples,
- the real measured window duration is close to `5 seconds`,
- the average value is near zero, which is expected for a balanced sinusoidal signal over time,
- the min and max values are stable across windows,
- the generation and buffering process is repeatable.

9.13 Why the Average Near Zero Is Important

The aggregate function required by the project is the average over a time window. Since the current signal is a sum of sinusoids centered around zero, the average over a sufficiently complete window should be close to zero.

The observed values, such as `0.0037` and `-0.0000`, are consistent with this expectation and therefore support the correctness of the implementation.

9.14 What Was Actually Tested in Phase 3

Phase 3 tested the following:

- the board still boots correctly after the new signal logic is added,
- the firmware generates real dynamic sample values,
- the sampling timing remains stable,
- sample windows fill to the expected size,
- simple per-window statistics are coherent and repeatable.

Phase 3 did **not** yet test:

- FFT correctness,
- dominant-frequency estimation,
- adaptive sampling changes,
- real MQTT publication from the ESP32,
- real LoRaWAN transmission.

9.15 Issues and Small Observations

Two minor observations remained after the successful run:

1. One log line still printed `Phase 2 tasks created successfully` even though the firmware was already in Phase 3.
2. The MQTT placeholder payload still contained the JSON field `"phase":2,"state":"skeleton"`.

These are only labeling leftovers. They do not affect the correctness of the virtual sensor implementation.

9.16 Summary of Phase 3 Results

Check	**Status**
Virtual sinusoidal signal implemented	Passed
Stable periodic sampling at 50 Hz observed	Passed
First samples show a non-constant waveform	Passed
Window buffer fills correctly with 250 samples	Passed
Window duration close to 5 seconds	Passed
Window average remains close to zero	Passed
Repeated windows show stable min/max values	Passed
No crash, panic, or watchdog reset observed	Passed

10 Phase 4: Sampling Benchmark

10.1 Goal of Phase 4

The goal of Phase 4 was to measure the highest stable sampling frequency that the current board and firmware could support before FFT and adaptive sampling were implemented.

At this stage, the question was no longer whether the firmware could boot or generate a signal. Instead, the focus was on timing performance.

The practical question of Phase 4 was:

> What is the maximum stable sampling rate of the current ESP32 firmware when the system is required to generate samples on time, feed them through the queue pipeline, and avoid queue drops and timing violations?

10.2 Why This Phase Matters

The project later requires adaptive sampling. To judge whether adaptive sampling saves time, energy, or communication cost, a baseline must first be measured.

Phase 4 provides that baseline by identifying:

- a safe starting frequency,
- the point where timing problems begin,
- the approximate upper operating region of the current firmware.

10.3 Benchmark Method

The firmware was switched into a dedicated `sampling_benchmark` mode.

In this mode:

- the signal input task generates the same virtual signal used in Phase 3,
- the signal processing task switches into a simple drain mode,
- one benchmark stage runs for `4 seconds`,
- several target sampling frequencies are tested in sequence,
- timing statistics are printed at the end of each stage.

Frequencies tested

The benchmark sweep tested the following target rates:

- `50 Hz`
- `100 Hz`
- `200 Hz`
- `250 Hz`
- `500 Hz`
- `1000 Hz`

Metrics recorded

For each stage, the firmware recorded:

- target frequency,
- achieved frequency,
- number of generated samples,
- number of consumed samples,

- queue drops,
- deadline misses,
- minimum interval between samples,
- maximum interval between samples,
- worst observed lateness.

Stability rule

In the current firmware, a benchmark stage is considered **stable** only if both of the following conditions are true:

1. queue drops = `0`
2. deadline misses = `0`

This is a strict criterion. It means a stage may achieve the requested average rate but still be classified as unstable if even a small number of timing violations occur.

10.4 Issue Encountered Before the Final Benchmark Run

The first attempt at Phase 4 did not complete correctly.

Observed problem

The benchmark crashed when it reached `200 Hz`.

Cause

The original implementation used `vTaskDelayUntil()` with a delay based on FreeRTOS ticks. Because the system tick rate is much lower than the desired high-frequency sample periods, some benchmark stages produced a zero-tick delay, which caused an assertion failure.

Fix applied

The benchmark timing logic was changed so that Phase 4 uses microsecond-based scheduling with `esp_timer_get_time()` instead of relying on tick-only periodic delays.

A small startup-log issue was also corrected so that the firmware now reports the correct benchmark mode at boot.

10.5 Observed Board Logs

The corrected benchmark firmware booted with the following key lines:

```
I (...) app_main: Firmware booting for adaptive-sampling-node in
    sampling_benchmark mode
I (...) signal_input: Signal input module ready in sampling_benchmark mode
I (...) signal_processing: Signal processing module ready in benchmark drain
    mode
I (...) signal_input: Benchmark stage 1/6 started at 50.0 Hz
```

These lines confirm that the benchmark mode was entered correctly and that the sample-processing task was simplified into a drain-only mode so that the timing measurement focused on acquisition throughput.

10.6 Measured Results

The following benchmark result lines were observed on the ESP32 board:

```
Benchmark result | stage=1/6 target=50.0Hz achieved=50.25Hz
samples=201 consumed=201 drops=0 misses=0 ... stable=yes

Benchmark result | stage=2/6 target=100.0Hz achieved=100.25Hz
samples=401 consumed=401 drops=0 misses=1 ... stable=no

Benchmark result | stage=3/6 target=200.0Hz achieved=200.25Hz
samples=801 consumed=801 drops=0 misses=2 ... stable=no

Benchmark result | stage=4/6 target=250.0Hz achieved=250.25Hz
samples=1001 consumed=1000 drops=0 misses=12 ... stable=no

Benchmark result | stage=5/6 target=500.0Hz achieved=500.25Hz
samples=2001 consumed=2000 drops=0 misses=26 ... stable=no

Benchmark result | stage=6/6 target=1000.0Hz achieved=1000.00Hz
samples=4000 consumed=3999 drops=1 misses=55 ... stable=no
```

10.7 Result Table

Target	Achieved	Drops	Misses	Stable	Observation
50 Hz	50.25 Hz	0	0	Yes	Clean run under the strict rule
100 Hz	100.25 Hz	0	1	No	Very close to stable, but one deadline miss occurred
200 Hz	200.25 Hz	0	2	No	Throughput was maintained, but timing violations appeared
250 Hz	250.25 Hz	0	12	No	Task watchdog warnings began to appear
500 Hz	500.25 Hz	0	26	No	Repeated watchdog warnings and larger timing instability
1000 Hz	1000.00 Hz	1	55	No	Severe timing instability and one queue drop

10.8 Interpretation of the Results

The benchmark reveals an important distinction between *average throughput* and *stable timing*.

The achieved frequency remained close to the target value even at high rates. However, the strict stability rule also requires zero deadline misses and zero queue drops. Under that rule:

- `50 Hz` is stable,
- `100 Hz` is already unstable,
- all higher tested rates are unstable,
- watchdog warnings begin at `250 Hz` and above.

This means that the highest stable frequency among the tested values is:

50 Hz

10.9 What Can Be Concluded From This

Phase 4 does **not** show that the board is incapable of generating samples above `50 Hz`. In fact, it clearly generated and consumed data at much higher average rates.

What it does show is that:

- `50 Hz` is the highest tested frequency that satisfies the current strict stability definition,
- the true threshold lies somewhere between `50 Hz` and `100 Hz`,
- watchdog warnings make `250 Hz` and above unsuitable for a clean baseline configuration in the current implementation.

This gives a justified baseline for the next phases.

10.10 Summary of Phase 4 Results

Check	**Status**
Benchmark mode implemented and executed on the board	Passed
Six target sampling rates tested successfully	Passed
Corrected microsecond-based scheduling prevented the earlier assert failure	Passed
Stable operating point found under strict rule	Passed
Maximum stable tested sampling frequency identified as `50 Hz`	Passed
Instability and watchdog warnings documented for higher rates	Passed

11 Phase 5: Frequency Analysis and Dominant-Frequency Detection

11.1 Goal of Phase 5

The goal of Phase 5 was to replace the placeholder frequency result with a real spectral analysis stage.

After Phase 4, the firmware already had:

- a stable virtual sensor mode at `50 Hz`,
- a complete `250`-sample window every `5 seconds`,
- a queue pipeline capable of forwarding one frequency-analysis result per window.

The remaining question was:

> Can the firmware analyse the completed signal window and correctly identify the dominant frequency component of the generated signal?

11.2 Expected Spectral Result

The input signal used in Phase 5 remained the same as in Phase 3:

$$2\sin(2\pi \cdot 3t) + 4\sin(2\pi \cdot 5t)$$

From this expression, the expected frequency-domain behavior is:

- one component near `3 Hz`,
- one component near `5 Hz`,
- the `5 Hz` component should be dominant because it has the larger amplitude.

11.3 Implementation Approach

The signal-processing module was upgraded so that each completed window is now analysed in the frequency domain.

The main steps performed in Phase 5 are:

1. collect `250` samples into a complete window,
2. compute the window mean,
3. remove the DC offset by centering the samples around that mean,
4. evaluate the magnitude of each spectral bin,
5. find the dominant peak,
6. send the resulting dominant frequency and peak magnitude to the FFT queue.

Important implementation note

The project requirement refers to FFT analysis. In the current Phase 5 implementation, the firmware performs a direct DFT-style spectral computation over the window rather than an optimized FFT library call.

This still produces the required frequency result correctly for the current `250`-sample validation case, but it is more computationally expensive than a true optimized FFT. For this reason, the functional result is validated in Phase 5, while further optimization remains possible later if needed.

11.4 Why the Stable 50 Hz Mode Was Used

The benchmark results from Phase 4 showed that `50 Hz` is the highest tested sampling rate that satisfied the strict stability rule.

For this reason, Phase 5 intentionally returned to the stable virtual-sensor mode instead of performing spectral analysis during the benchmark mode.

This was important because Phase 5 should validate the correctness of the frequency result, not the limits of the scheduler.

11.5 Observed Board Logs

The board booted in virtual-sensor mode and produced the following key lines:

```
I (...) app_main: Firmware booting for adaptive-sampling-node in virtual_sensor
    mode
I (...) signal_processing: Signal processing module ready with 250-sample
    spectral window
```

The first fully analysed window then produced:

```
Window 0 ready | samples=250 avg=0.0030 min=-5.6716 max=5.6716 duration=4.98s
FFT result | window=0 dominant=5.00 Hz peak=1.9996 expected_3Hz=0.9998
    expected_5Hz=1.9996
FFT peak[0] | bin=25 freq=5.00 Hz magnitude=1.9996
FFT peak[1] | bin=15 freq=3.00 Hz magnitude=0.9998
FFT peak[2] | bin=5 freq=1.00 Hz magnitude=0.0030
```

Subsequent windows showed equally strong consistency, for example:

```
FFT result | window=1 dominant=5.00 Hz peak=2.0000 expected_3Hz=1.0000
    expected_5Hz=2.0000
FFT result | window=2 dominant=5.00 Hz peak=2.0000 expected_3Hz=1.0000
    expected_5Hz=2.0000
FFT result | window=3 dominant=5.00 Hz peak=2.0000 expected_3Hz=1.0000
    expected_5Hz=2.0000
```

11.6 Interpretation of the Logs

These logs confirm the following:

- the dominant frequency is consistently detected at `5.00 Hz`,

- the secondary component is consistently detected at `3.00 Hz`,
- the relative magnitudes are stable from window to window,
- the result is exactly aligned with the structure of the generated test signal.

This is the key functional success of Phase 5.

11.7 Representative Spectrum Graph

Figure 1 shows a simple representative spectral view for one analysed window. The values are taken from the measured runtime output.

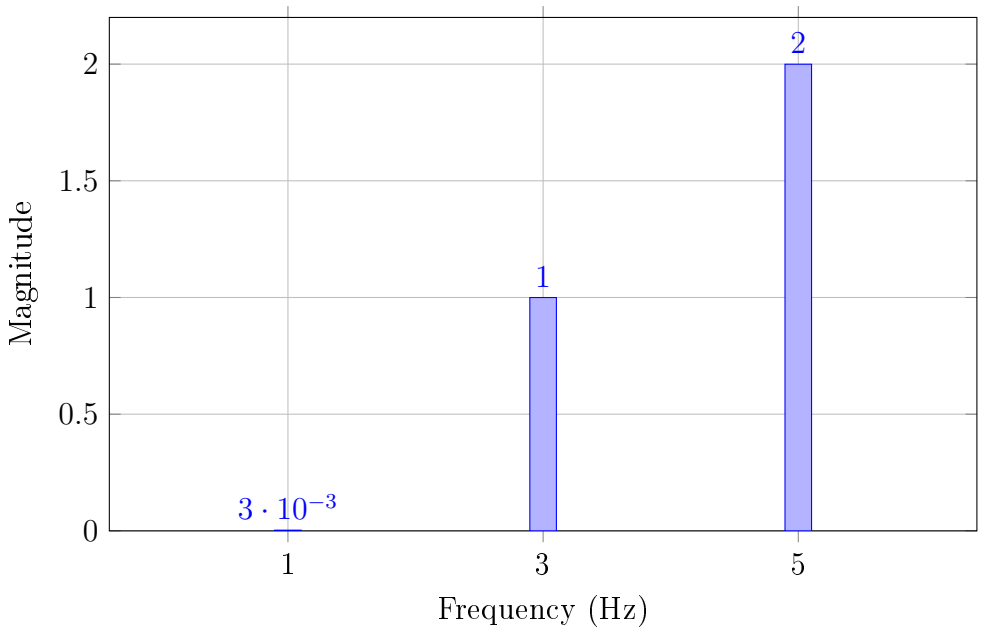

Figure 1: Representative spectral magnitudes from Phase 5 for one analysed window. The `5 Hz` component is dominant and the `3 Hz` component is clearly visible, matching the generated signal.

11.8 Why the Graph Is Useful

The graph makes the result easy to understand visually:

- the `5 Hz` bar is the highest,
- the `3 Hz` bar is clearly present but smaller,
- the low residual component near `1 Hz` is negligible.

This gives a compact visual confirmation that the spectrum behaves exactly as expected from the signal definition.

11.9 Summary of Phase 5 Results

Check	Status
Real spectral analysis replaced the placeholder FFT result	Passed
Dominant frequency detected consistently at `5.00 Hz`	Passed
Secondary component detected consistently at `3.00 Hz`	Passed
Measured magnitudes remained stable across repeated windows	Passed
Frequency result was forwarded correctly through the FFT queue	Passed
Board remained stable during repeated window analysis	Passed

12 Phase 6: Adaptive Sampling

12.1 Goal of Phase 6

The goal of Phase 6 was to make the system react to the dominant frequency that was identified in Phase 5.

Until the end of Phase 5, the firmware could:

- generate the signal,
- build complete windows,
- identify the dominant frequency correctly.

However, the sampling rate itself still remained fixed.

The practical question of Phase 6 was:

> Can the firmware use the dominant frequency estimate to change the sampling rate at runtime while preserving correct spectral analysis on later windows?

12.2 Adaptive Policy Implemented

Phase 6 introduced a conservative adaptive rule.

The implemented policy is:

- requested sampling rate = `8 x dominant frequency`
- the result is rounded to `5 Hz` steps
- the final rate is clamped between `20 Hz` and `50 Hz`

Why this policy was chosen

This policy was intentionally conservative for two reasons:

- Phase 4 showed that the current stable operating region is limited, so the adaptive policy should not exceed the safe upper bound.
- the purpose of Phase 6 is to prove correct adaptation behavior first, not to maximize the sampling rate aggressively.

For the current signal, where the dominant frequency is `5 Hz`, the policy gives:

$$8 \times 5 = 40\,\mathrm{Hz}$$

So the expected first adaptation is from `50 Hz` down to `40 Hz`.

12.3 Implementation Changes

Phase 6 required coordinated changes in several modules.

12.3.1 Signal Input Module

The signal-input task was updated so that:

- it no longer assumes a fixed sampling rate forever,
- it reads the current adaptive sampling frequency from shared state,
- it applies the new frequency at runtime,
- it logs when a new sampling rate becomes active.

12.3.2 Sampling Control Module

The sampling-control task was upgraded from a placeholder logger into a real decision stage.

It now:

- reads FFT results from the queue,
- computes the requested sampling rate,
- compares it to the current rate,
- either updates the rate or holds the existing one,
- logs each decision clearly.

12.3.3 Signal Processing Module

The signal-processing stage also required one important correction.

When the sampling rate changes during an unfinished window, the remaining part of that window no longer belongs to a single uniform sampling condition. If those mixed samples are analysed together, the spectrum becomes distorted.

To prevent this, the processing task was updated so that:

- if a sampling-rate change is detected inside a partial window,
- that partial window is discarded,
- a new clean window starts at the new sampling rate.

This behavior is essential for correctness.

12.4 Observed First Adaptation

The first important Phase 6 result appeared immediately after the first `50 Hz` window:

```
Window 0 ready | fs=50.0Hz samples=250 avg=0.0041 min=-5.6139 max=5.6139
    duration=4.97s
Adaptive sampling update | window=0 dominant=5.00 Hz peak=2.0000 previous_fs
    =50.0 Hz new_fs=40.0 Hz
Applying adaptive sampling frequency 40.0 Hz
```

Interpretation

This confirms that:

- the FFT result was used by the control logic,
- the policy computed the correct new rate,
- the input task adopted the new rate in runtime.

12.5 Important Transition Problem and Fix

During the first Phase 6 implementation, a real problem appeared.

Observed issue

After the rate changed from `50 Hz` to `40 Hz`, the next window initially mixed samples from two different sampling rates. This produced incorrect dominant-frequency estimates in later windows.

Corrective behavior

After the fix, the firmware produced the following warning:

```
Sampling-rate transition inside window 1 after 28/250 samples (50.0 Hz -> 40.0
    Hz);
discarding partial window and restarting
```

Why this is correct

This warning is not a failure. It is the correct protective behavior.

By discarding the partial mixed-rate window, the firmware ensures that every analysed window corresponds to only one true sampling frequency.

12.6 Clean 40 Hz Windows

After the transition guard was added, the next windows became clean and consistent.

The first fully clean adaptive window produced:

```
Window 1 ready | fs=40.0Hz samples=200 avg=0.0000 min=-5.6138 max=5.6138
    duration=4.98s
Adaptive sampling hold | window=1 dominant=5.00 Hz peak=2.0000 fs=40.0 Hz
FFT result | window=1 dominant=5.00 Hz peak=2.0000 expected_3Hz=1.0000
    expected_5Hz=2.0000
```

Subsequent windows continued with the same behavior, for example:

```
Window 2 ready | fs=40.0Hz samples=200 avg=-0.0000 min=-5.6138 max=5.6138
    duration=4.98s
Adaptive sampling hold | window=2 dominant=5.00 Hz peak=2.0000 fs=40.0 Hz

Window 3 ready | fs=40.0Hz samples=200 avg=-0.0000 min=-5.6139 max=5.6139
    duration=4.98s
Adaptive sampling hold | window=3 dominant=5.00 Hz peak=2.0000 fs=40.0 Hz
```

12.7 Interpretation of the Adaptive Results

These logs confirm the main success criteria of Phase 6:

- the system performed a real sampling-rate change at runtime,
- the new rate matched the adaptive policy,
- a mixed transition window was detected and safely discarded,
- later windows were analysed at a clean `40 Hz`,
- the detected dominant frequency remained correct at `5.00 Hz`,
- the controller then correctly held the rate steady at `40.0 Hz`.

12.8 Why 200 Samples Is the Correct New Window Size

The signal-processing stage keeps the window duration at `5 seconds`. When the sampling rate changes to `40 Hz`, the expected number of samples in a full window becomes:

$$40 \times 5 = 200$$

The measured logs show exactly `200` samples per clean adaptive window, which confirms that the dynamic window resizing works correctly.

12.9 Summary of Phase 6 Results

Check	Status
Adaptive sampling policy implemented	Passed
First adaptive update changed sampling from `50 Hz` to `40 Hz`	Passed
Input task applied the new sampling frequency at runtime	Passed
Mixed-rate partial window was detected and discarded correctly	Passed
Later windows ran cleanly at `40 Hz` with `200` samples	Passed
Dominant frequency remained correct after adaptation	Passed
Controller correctly held the stable adaptive rate at `40 Hz`	Passed

13 Phase 7: Aggregate Computation

13.1 Goal of Phase 7

The goal of Phase 7 was to turn the analysed window into a real aggregate data object that can later be transmitted through MQTT or LoRaWAN.

At the end of Phase 6, the firmware already knew:

- the current adaptive sampling rate,
- the dominant frequency of each completed window,
- the average value of the window internally.

However, this information was not yet packaged into a dedicated result that other modules could consume safely.

The practical question of Phase 7 was:

> Can the firmware compute and store a per-window aggregate result and expose it clearly in the runtime pipeline without breaking the already working adaptive signal path?

13.2 Aggregate Structure Used

The aggregate result already existed as a data type in the firmware design, but before Phase 7 it was not being used in practice.

The active aggregate contains:

- window identifier,
- window start timestamp,
- window end timestamp,

- sample count,
- sampling frequency,
- dominant frequency,
- average value of the window.

This set of fields is enough for the later communication phases because it captures both the main statistical value required by the assignment and the important context in which that value was measured.

13.3 Implementation Changes

Phase 7 required three practical changes.

13.3.1 Signal Processing Module

The signal-processing stage was extended so that, whenever a window is completed:

- the spectral result is still produced as before,
- a matching aggregate object is created,
- the aggregate is pushed into the dedicated aggregate queue,
- the queue keeps the newest result if it ever becomes full.

This means that a completed window now produces two useful outputs:

- FFT information for adaptive control,
- aggregate information for later communication.

13.3.2 Metrics Module

The metrics task was upgraded so that it now:

- reads aggregate results from the aggregate queue,
- stores the newest aggregate snapshot in shared project state,
- tracks how many aggregate results have been processed,
- logs the aggregate content explicitly.

13.3.3 Display and Supervisor Logs

The display heartbeat and supervisor heartbeat were also improved so that they show:

- the current aggregate count,
- the latest average value,
- the latest aggregate window identifier.

These logs make Phase 7 easy to verify directly from the serial monitor.

13.4 Observed First Aggregate Result

The first successful aggregate appeared immediately after the first completed `50 Hz` window:

```
Window 0 ready | fs=50.0Hz samples=250 avg=0.0041 min=-5.6139 max=5.6139
    duration=4.97s
Aggregate result | window=0 fs=50.0Hz samples=250 avg=0.0041 dominant=5.00 Hz
```

Interpretation

This confirms that:

- the average value was not only computed internally,
- it was packaged into a real aggregate object,
- the aggregate queue and metrics consumer were both functioning.

13.5 Aggregate Results After Adaptation

After the transition from `50 Hz` to `40 Hz`, the aggregate pipeline continued to work correctly on the clean adaptive windows.

Representative lines from the runtime log were:

```
Aggregate result | window=1 fs=40.0Hz samples=200 avg=-0.0000 dominant=5.00 Hz
Aggregate result | window=2 fs=40.0Hz samples=200 avg=-0.0000 dominant=5.00 Hz
Aggregate result | window=3 fs=40.0Hz samples=200 avg=-0.0000 dominant=5.00 Hz
```

The display and supervisor logs also reflected the same information:

```
Display heartbeat | mode=1 events=0xFF fs=40.0Hz dominant=5.00Hz avg=-0.0000
    window=2
Supervisor heartbeat | ... windows=3 aggregates=3 ... fs=40.0Hz dominant=5.00Hz
    avg=-0.0000 updates=1
```

13.6 Why These Results Are Correct

The generated signal is centered around zero, so over a full clean window the average should remain very close to zero.

The measured aggregate values satisfy that expectation:

- the first window average was very small, about `0.0041`,
- later clean windows at `40 Hz` produced averages of approximately `0.0000`,
- the dominant frequency remained stable at `5.00 Hz`.

This is the correct behavior. It shows that the aggregate computation is consistent with both the signal shape and the already validated frequency analysis.

13.7 Relationship to Later Communication Phases

Phase 7 completes the internal data path needed before real communication is implemented.

At this point, the system can already produce a clean aggregate that includes:

- the required window average,
- the number of samples used,
- the adaptive sampling rate,
- the dominant frequency of the same window.

This means the MQTT and LoRaWAN stages no longer need to invent placeholder content forever. In the next phases, they can be converted to publish real aggregate payloads.

13.8 Summary of Phase 7 Results

Check	**Status**
Per-window aggregate result created from each completed window	Passed
Aggregate queue is now actively used	Passed
Metrics task consumes and stores aggregate results correctly	Passed
Latest average value appears in display and supervisor logs	Passed
Aggregate count increases correctly during runtime	Passed
Aggregate values remain correct after adaptive sampling changes	Passed
Average stays near zero for clean windows of the centered virtual signal	Passed

14 Conclusion

Phase 1, Phase 2, Phase 3, Phase 4, Phase 5, Phase 6, and Phase 7 were completed successfully.

At the end of these seven phases, the following were confirmed:

- the ESP32 board and serial connection work correctly,
- the development environment can build and upload firmware,
- runtime monitoring through serial output works correctly,
- the MQTT broker is operational,
- MQTT communication works both locally and over the local network.

- the firmware architecture boots correctly as a modular application,
- all planned subsystems are separated and initialized correctly,
- the basic FreeRTOS pipeline structure is active,
- the board can generate the required virtual signal in a stable and repeatable way,
- the firmware can store and validate real sample windows,
- the current strict maximum stable tested sampling rate is `50 Hz`,
- timing instability begins between `50 Hz` and `100 Hz` under the current benchmark rule,
- watchdog warnings appear at higher benchmark rates,
- the dominant spectral component is correctly detected at `5 Hz`,
- the secondary spectral component is correctly detected at `3 Hz`,
- the sampling rate can now adapt automatically at runtime,
- the current adaptive policy reduces the rate from `50 Hz` to `40 Hz` for the present signal,
- the system preserves correct frequency detection after the adaptive transition,
- each completed window now produces a real aggregate result,
- the aggregate includes the window average, sampling frequency, and dominant frequency,
- the aggregate pipeline remains correct after adaptation.

This means the project is now ready for **Phase 8**, where the MQTT path should stop using placeholder payloads and begin publishing the real aggregate results generated in Phase 7.

The next phase should focus on:

- replacing the placeholder MQTT payload with a payload built from the real aggregate result,
- sending the aggregate average, sampling rate, and dominant frequency over MQTT,
- validating that the broker receives the real payload from the board,
- keeping the adaptive signal pipeline stable while communication is enabled.

15 Appendix: Important Commands Used in Phases 1 to 7

```
& "$HOME\.platformio\penv\Scripts\pio.exe" run
```

```
& "$HOME\.platformio\penv\Scripts\pio.exe" run -t upload --upload-port COM3
```

```
& "$HOME\.platformio\penv\Scripts\pio.exe" device monitor -p COM3 -b 115200
```

```
& "C:\Program Files\mosquitto\mosquitto_sub.exe" -h localhost -t stage1/test -v
```

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -h localhost -t stage1/test -m
    "hello from stage1"
```

```
& "C:\Program Files\mosquitto\mosquitto_sub.exe" -h 192.168.1.22 -t stage1/test
    -v
```

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -h 192.168.1.22 -t stage1/test
    -m "hello over lan"
```

```
cd firmware/esp32_node
```

```
& "$HOME\.platformio\penv\Scripts\pio.exe" run
```

```
& "$HOME\.platformio\penv\Scripts\pio.exe" run -t upload --upload-port COM3
```

```
& "$HOME\.platformio\penv\Scripts\pio.exe" device monitor -p COM3 -b 115200
```